[Dung Vu] Toolchain & Cross-Compilation

- **Section**: Embedded Linux
- **Topic**: Toolchain & Cross-Compilation

---

1. What is Cross-compiling Toolchain

Cross-compiling toolchain (or *cross-toolchain*) is a set of tools that allows to build source code into binary code for a target platform different than the one where the build takes place.

- Different CPU architecture
- Different ABI (Application Binary Interface)
- Different operating system
- Different C library

Three machines involved in the build process

- **build** machine, where the build takes place
- **host** machine, where the execution takes place
- **target** machine, for which the programs generate code

Native toolchain: *build == host == target*

Cross-compilation toolchain: *build == host != target*

2. Cross-compiling Toolchain Components

There are 4 core components in Linux cross-compiling toolchain

- **Compiler** (`gcc`): turns Source Code -> Assembly Code
- **Binary utilities** (GNU binutils): collection of binary tools
  Main tools:
  - Assembler (`as`): turns Assembly Code -> Object Code
  - Linker (`ld`): turns Object Code + C Library -> Executable Binary

  Debugging/analysis tools and other tools: addr2line, ar, c++filt, gold, gprof, nm, objcopy, objdump, ranlib, readelf, size, strings, strip
- **C library**: provide a standard abstraction layer that performs basic tasks
  - Linux: `glibc`, `eglibc`, `uClibc` or `musl`
  - Without OS: `newlib`, `dietlibc` or even none
- **Linux kernel headers**: provides the API to the kernel features

3. How the Components Chained Together

Building a cross-toolchain involves a classic chicken-and-egg problem:

- the **final compiler** needs the **C library**, to know how to use it, and the **binutils**
- building the **C library** requires a **core pass 2 compiler** and the **binutils**
- the **core pass 2 compiler** needs the **C library headers** and **start files**, to know how to use the **C library**, and the **binutils**
- building the **start files** requires a **core pass 1 compiler**, **the kernel headers** and the **binutils**
- the **core pass 1 compiler** needs the **binutils**

Which turns in this order to build the components:

Step	Components	Required
1	host math libraries (GMP, MPFR, MPC)	*Host compiler only*
2	binutils	*Host compiler only*
3	core pass 1 compiler	*Math libraries, binutils*
4	kernel headers	*Target Linux kernel source*
5	C library headers and start files	*Core Pass 1 Compiler, binutils, Kernel Headers*
6	core pass 2 compiler	*binutils, C Library Headers, CRT Start Files*
7	complete C library	*Core Pass 2 Compiler, binutils, Kernel Headers*

Step	Components	Required
8	final compiler	*Complete C Library, binutils*

4. How to Get a Cross-compilation Toolchain

Pre-built

- From your distribution. Ubuntu and Debian have numerous cross-compilers readily available.
- From various organization:
  - Linaro provides ARM and AArch64 toolchains
  - Mentor provides a few free Sourcery CodeBench toolchains
  - Imagination provides MIPS toolchains, etc.

Built it yourself

- Crosstool-NG, tool specialized in building cross-compilation toolchain. By far the most configurable/versatile.
- Embedded Linux build systems generally all know how to build a cross-compilation toolchain: Yocto/OpenEmbedded, Buildroot, OpenWRT, etc.

Ref

https://crosstool-ng.github.io/docs/toolchain-construction/

https://bootlin.com/pub/conferences/2016/elce/petazzoni-toolchain-anatomy/petazzoni-toolchain-anatomy.pdf

---

Revision #1
Created 2026-10-21 07:14:51 UTC by Dung Vu
Updated 2026-10-21 07:15:35 UTC by Dung Vu